POSA / Unplugged AI — Codex Planning Handoff

0. Purpose of This Handoff

This document summarizes the current planning conversation for building **POSA — Portable Offline Survival Assistant**, a free/open-source Android-first survival app built by **Unplugged AI**.

The goal is to give Codex enough context to continue planning and implementation without repeating old work, making wrong assumptions, or overbuilding features that were intentionally delayed.

This handoff should be treated as one planning input. The user intends to upload this together with two related branch conversations for broader context.

---

1. Project Goal and Strategic Context

1.1 Main Project Goal

Build **POSA — Portable Offline Survival Assistant**:

> A free, open-source, Android-first offline survival app for hikers, campers, preppers, outdoors users, CERT-style groups, and eventually broader field users.

The product should provide:

- Offline maps
- Offline survival/bushcraft content
- Guided “chat with manuals” / RAG-style Q&A
- Basic survival checklists
- Field tools
- User kit/tool context
- Provenance/citations for guidance
- Open-source community traction

The app should be useful even without AI, but should be architected so local RAG and optional local models can be added cleanly.

1.2 Strategic Company Goal

Use POSA as a public/open-source trust layer for **Unplugged AI**.

Unplugged AI’s larger positioning:

> The go-to company for decision-grade intelligence at the edge.

The broader commercial/government thesis:

- Unplugged AI is already pursuing government/defense/public safety opportunities.
- Existing interest includes TradeWinds, Border Patrol, intelligence community, and potential government pilot work.
- POSA should help prove that Unplugged AI can ship useful offline-first field software.
- POSA should build public credibility, GitHub stars, user feedback, and field validation.
- POSA should support future fundraising by showing bootstrapped traction and a working public product.

POSA should **not** become the entire company identity. It should be one public product that proves the platform.

---

2. Current State

2.1 No Implementation Yet From This Chat

As of this conversation:

- No POSA repo has been created in this chat.
- No Android project has been initialized in this chat.
- No files, scripts, packages, or source code have been finalized here.
- This is a planning handoff, not a code handoff.

2.2 Existing Related Work / Background

Unplugged AI already has adjacent work around:

- Offline RAG
- Local small language models
- Edge AI
- Government pilots
- ATAK/government field workflows
- Survival/offline assistant concepts from earlier planning

Important: Codex should not assume POSA is starting from a mature existing codebase unless the related branch conversations provide one.

2.3 Original Concept Evolution

The original idea was:

> Build an open-source RAG app for campers and hikers using basic bushcraft/survival manuals.

The concept evolved back toward an older POSA-style idea:

> A broader offline survival companion with maps, checklists, tools, and chat/manual retrieval.

The final MVP direction is **not just a chatbot**. It is an offline field companion where RAG is one part of the user experience.

---

3. Brand Architecture Decisions

3.1 Product Name

Chosen product name:

POSA

Portable Offline Survival Assistant

Brand line:

> Built by Unplugged AI

Recommended public-facing format:

```
POSA
Portable Offline Survival Assistant
Built by Unplugged AI
```

3.2 Brand Separation

Decision:

- **POSA** should be the public/open-source product brand.
- **Unplugged AI** should remain the company/platform brand.
- POSA should build trust and community without boxing Unplugged AI into only being a prepper/survival app company.

Brand hierarchy:

```
Unplugged AI
│
├── POSA
│   Portable Offline Survival Assistant
│   Free/open-source civilian survival app
│
├── Unplugged Edge / Edge Intelligence Platform
│   Commercial offline intelligence platform
│
├── TAK Assistant / FieldOps
│   Government/ATAK/SOP decision-support product
│
└── Custom Knowledge Packs
    Paid customer-specific packs and deployments
```

3.3 Repo Naming Ideas

Repo name options discussed:

- `posa`
- `posa-survival`
- `posa-app`
- `posa-offline`
- `portable-offline-survival-assistant`
- `unpluggedai/posa`

Preferred GitHub structure:

```
github.com/unpluggedai/posa
```

Reason:

- POSA remains the product name.
- Unplugged AI still receives attribution.
- Easy to explain publicly.

3.4 Naming Warning

POSA is not necessarily unique as an acronym.

Known unrelated uses include:

- Pattern-Oriented Software Architecture
- Point-of-Sale Activation

- Other unrelated acronym uses

Action needed later:

- Do basic trademark/name availability check.
- Use full phrase “Portable Offline Survival Assistant” often.
- Avoid assuming `posa` alone is legally clean.

---

4. Final Strategic Decision

4.1 Build POSA

Final direction:

> Build POSA as an Android-first, open-source app.

The app should be free/open source and should act as a public proof-of-capability for Unplugged AI.

4.2 Android First

Decision:

- Focus on Android first.
- Open-source the Android app codebase.
- Do not try to build Android, iOS, desktop, web, and CLI simultaneously.

Rationale:

- Faster to reach a working product.
- Simpler Codex scope.
- Better fit for outdoors/prepper users who often prefer Android because of device variety, affordability, customization, rugged devices, sideloading, and F-Droid-style distribution.
- Government/rugged/field devices are often Android-based.
- Android has better alignment with open-source distribution and offline customization.

4.3 Open Source Scope

Decision:

- Open-source the POSA Android app.
- Do not create a separate multi-OS open-source framework first.
- Do not start by building in Python and trying to bridge it into Android.

Reasoning:

- Python-to-Android stacks such as Kivy/BeeWare were considered but not recommended for a polished field app.
- Native or mobile-first development will likely save time and friction later.
- Android-first gives a cleaner path to performance, sensors, maps, permissions, offline storage, and polished UX.

---

5. Strategic Rationale

5.1 Why Build an Open-Source Product?

The intended benefits:

- Get GitHub stars.
- Get public users.
- Get field feedback from real outdoors/prepper users.
- Build a community around POSA and Unplugged AI.
- Create public evidence of execution.
- Show investors that Unplugged AI can ship.
- Use POSA as field validation before/alongside government pilots.
- Strengthen government conversations by showing a working offline edge product.
- Position Unplugged AI as credible and bootstrapped.

5.2 Government/Funding Thesis

The user is already in or near the government arena:

- TradeWinds-related work.
- Inbound from Border Patrol.
- Intelligence community interest.
- Roughly one month from a first real government pilot, per user's expectation.

POSA is not expected to directly win government contracts by itself.

Instead, POSA should:

- Demonstrate the architecture publicly.
- Show real-world usage.
- Build trust.
- Create a public credibility layer.
- Prove offline-first UX and deployment ability.
- Support fundraising after government pilot traction.

Future pitch framing:

> POSA is the free/open-source public survival assistant. Unplugged AI builds hardened, mission-specific decision-grade intelligence systems for edge environments.

5.3 Competition Strategy

User's strategic idea:

> Release a free/open-source version that meets or exceeds baseline competitor functionality, while Unplugged AI's proprietary version remains faster, smaller, more deployable, and more mission-specific.

This is an open-core strategy.

The goal is not to give away the whole moat. The goal is to raise the baseline and make Unplugged AI look like the team defining the category.

---

6. Moat Strategy

6.1 What to Open Source

Open-source these layers:

- Android app shell
- Basic POSA UX
- Offline maps integration
- Basic guide/manual content
- Basic checklists
- Local notes
- Basic tools
- Basic local search
- Pack manifest format
- Basic content packs
- Provenance UI pattern
- Import/export for packs
- Documentation
- Contribution guidelines
- Basic public testing workflows

These create trust, adoption, and community.

6.2 What Not to Open Source Initially

Keep proprietary/private:

- Government SOP packs
- Mission-specific doctrine packs
- ATAK plugin/integration
- Evaluation harness
- Model optimization recipes
- Fine-tuning recipes
- Safety classifiers / answer scoring
- Secure pack builder/signing infrastructure
- Admin/deployment tooling
- Rugged device optimization
- Customer-specific workflows
- Government-specific RAG pipelines
- Border Patrol / IC / defense-specific integrations
- Advanced local model runtime optimizations
- Proprietary edge intelligence platform

6.3 Actual Moat

The moat is not the existence of a survival app.

The moat is:

> Turning mission knowledge into validated, offline, source-grounded decision systems that run on constrained edge devices.

Additional moat areas:

- Smaller/faster local inference
- Better retrieval quality
- Better citation/provenance UX
- Better device optimization
- Curated/validated packs
- Government integrations
- Deployment/support capability
- Evaluation and reliability testing
- Rugged field experience

---

7. Market Gap Summary From Research

The research conclusion:

> The market gap is not “more content.” It is integration plus trustworthy UX.

Current tools tend to win one layer at a time:

- Trail Sense: field tools
- Organic Maps: clean offline navigation
- Meshtastic: off-grid radio/mesh
- Briar: phone-to-phone messaging
- Kiwix: offline libraries
- SurvivalRAG: cited local Q&A
- CyberDeck-style projects: attribution/license sidecars and local sync ideas

Very few products combine:

- Offline maps
- Field tools
- Checklists
- Offline survival content
- Source-grounded Q&A
- Provenance
- Pack management
- Polished mobile UX

The opportunity:

> Build a small, credible, mobile-first, provenance-driven FOSS survival app that integrates the best ideas from fragmented tools.

7.1 Key Research Gaps Identified

Big market gaps:

1. **Integration**
2. Existing products are fragmented.
3. Few combine maps, tools, guides, checklists, and RAG.
4. **Trust/provenance UI**
5. Most survival apps do not clearly show where advice comes from.
6. Emergency guidance needs visible source separation.
7. **Battery-aware AI**

8. Many local AI/RAG projects assume desktop/laptop/base station hardware.
9. Phone-first survival AI must be tiered and optional.
10. **Polished open-source mobile UX**
11. Many FOSS projects are powerful but feel like hacker tools.
12. POSA should feel like a field instrument.

---

8. Final MVP Direction

8.1 User's Current MVP

The user's chosen MVP scope:

- Android app
- Open source
- POSA branding
- Offline maps
- Guided chat/manual Q&A
- Checklists
- Tools/toolkit
- User's personal tools/gear inventory as context
- Provenance/source display
- Field notes
- Curated content packs first

8.2 What Was Explicitly Delayed

Delayed / not in initial MVP:

- Bluetooth mesh
- Full mesh networking
- Meshtastic bridge
- Wi-Fi Direct peer sharing
- LAN sync
- Multi-OS support
- iOS
- Desktop app
- Full open-ended user-created pack ecosystem
- Heavy local LLM as required feature
- Government/ATAK plugin
- Advanced signed field notes

- Community pack registry
- Cloud backend
- Large model hosting
- Complex admin console

8.3 Why Bluetooth Mesh Was Delayed

User decided:

> Bluetooth mesh should probably be left out.

Reasoning:

- Adds complexity.
- Not needed for first value proposition.
- Can distract from the core app.
- Hardware/protocol constraints create extra support burden.
- Better to build maps, checklists, tools, and manual chat first.

8.4 Current MVP Feature Set

Core MVP features:

1. Offline Maps

Must include:

- Offline map viewing
- Offline saved areas or downloaded map data
- Current GPS coordinates
- Waypoints
- Basic route/track or breadcrumb support if feasible
- Map-first field utility

2. Checklists

Must include:

- Basic survival checklists
- Pre-trip checklist
- Emergency checklist
- Gear/kit checklist
- Water checklist
- Fire checklist
- Shelter checklist
- Navigation checklist
- First-aid checklist

- Vehicle/bug-out checklist if time permits

3. Guided Chat With Manuals

Must include:

- User asks questions against installed manuals/content packs.
- Responses should be grounded in local content.
- Citations/provenance visible.
- Start with guided Q&A, not unrestricted chatbot.
- Should support confidence/provenance display.

Important: Avoid presenting AI as an oracle.

4. Tools / Toolkit

Must include:

- Basic field tools tab.
- User can track tools/gear they have.
- The app can use that inventory as context later.

Possible tools:

- Compass
- Flashlight
- Timer
- Coordinates
- Notes
- Waypoint saver
- Simple unit conversions
- Water needs estimator
- Fire/shelter checklist access
- Weather estimation only if offline/sensor-based and reliable

5. User Gear/Tool Inventory as Context

User idea:

> Let the user specify what tools they have, then integrate that into the context window.

Example:

If the user has:

- lighter
- ferro rod
- tarp

- water filter
- knife
- cordage

Then the app should tailor guidance:

- “Given you have a tarp and cordage...”
- “Since you do not have a water filter...”
- “Use the ferro rod only after preparing dry tinder...”

This should be implemented carefully and locally.

6. Provenance

Must include:

- Source title
- Source type
- Pack name
- Pack version
- License if known
- Last reviewed date if known
- Citation/snippet display
- Clear separation between source excerpt and model-generated synthesis

7. Field Notes

Recommended addition:

- User field notes
- Notes can link to location, checklist, guide card, or pack
- Notes remain local
- Future signed/peer-share functionality can build on this

---

9. UX Direction

9.1 App Should Feel Like a Field Instrument

The UX should not look like:

- A generic chatbot
- A dashboard of modules
- A hacker project
- A prepper novelty app
- A random PDF reader

It should feel like:

- Practical
- Clean
- Fast
- Offline-first
- Field-readable
- Trustworthy
- Minimal
- Battery-conscious

9.2 Proposed Four Tabs

Recommended top-level tabs:

1. **Map**
2. **Tools**
3. **Guide**
4. **Packs**

Alternative tab naming was not finalized, but this four-tab structure was strongly recommended.

9.3 Emergency UX Principles

The app should answer three field questions quickly:

1. Where am I?
2. What should I do now?
3. What sources support this guidance?

Avoid:

- Complex dashboards
- Long onboarding
- Requiring accounts
- Requiring cloud
- Hiding citations
- Making AI the main interface before trust exists

---

10. Architecture / Stack Choices

10.1 Platform

Decision:

- Android first.
- Open-source Android app.
- Do not build multi-platform first.

10.2 Language / Framework

Recommended direction:

- Kotlin / native Android is preferred.

Acceptable alternatives:

- Flutter
- React Native

Not recommended for main app:

- Python-first Android app through Kivy/BeeWare

Reason:

- Native/mobile-first stack is better for maps, sensors, permissions, storage, thermal/battery control, and polished UI.

10.3 Storage

Recommended:

- SQLite-first local storage.

Potential libraries/approaches:

- Native Android SQLite
- Room
- SQLCipher or equivalent for encrypted local storage if sensitive notes are included

Local storage should handle:

- Notes
- Checklists
- Gear/tool inventory

- Waypoints
- Pack manifests
- Installed pack metadata
- Search indexes
- User settings
- Recent queries
- Bookmarks

10.4 Search / Retrieval

MVP search:

- SQLite FTS / keyword search over installed content packs.

V1 retrieval:

- Hybrid search:
- Keyword search
- Compact semantic/vector search

Possible later options:

- Embedded vector index
- SQLite vector extension if viable
- ChromaDB for laptop/basecamp/local server tier
- Qdrant for richer document libraries/server tier

Guidance:

- Phone tier should be compact and efficient.
- Do not require Chroma/Qdrant on the phone unless testing proves it is acceptable.
- Keep retrieval modular.

10.5 RAG / Chat

Initial approach:

- Guided Q&A over curated local content.
- Avoid fully open-ended chatbot as MVP default.
- Show citations every time.
- Show confidence/provenance.
- Answers should be constrained to installed packs.

Possible answer pipeline:

```
User question
→ Detect active context:
```

```
   - installed packs
   - current checklist
   - map/location if available
   - user gear inventory
   - user selected scenario
→ Retrieve relevant chunks/cards
→ Rank results
→ Compose answer:
   - concise answer
   - step-by-step guidance
   - citations/source cards
   - warnings/when to seek professional help
→ Save query locally if user allows
```

10.6 Models

No model was finalized for the Android MVP.

Prior research suggested possible model classes:

Phone tier:

- Small GGUF model
- Phi-3 mini class
- Qwen3-4B class
- Other small local instruction models

Tablet/laptop tier:

- 7B–8B class models
- Example class: llama3.1:8b style setups

LAN/basecamp tier:

- Optional Ollama/LM Studio host
- Local network inference only
- Not required for MVP

Important:

- Do not hard-wire one model family.
- Model should be optional.
- App should remain useful without AI model installed.
- Retrieval/search should work without generation.

10.7 Maps

Offline maps are part of the MVP.

Competitive baseline from research:

- Organic Maps
- OsmAnd~

Expected capabilities eventually:

- Offline vector maps
- Hiking/cycling/road data if possible
- Contours/topographic layers if feasible
- GPX import/export
- On-map notes
- Waypoints
- Fast offline search
- Route/track display

MVP may start simpler:

- Offline map view
- Current location
- Saved waypoints
- Basic offline downloaded map area
- GPX import/export if feasible

Unresolved:

- Exact Android map SDK/library.
- Whether to use MapLibre, Mapsforge, Organic Maps-derived approach, OsmAnd components, MBTiles, PMTiles, or another stack.
- Need Codex research/planning before implementation.

10.8 Pack System

Content packs are central.

Each pack should have:

- Manifest
- Title
- Version
- Category
- Source/license metadata
- Last reviewed date

- Files/chunks/cards
- Optional embeddings/index
- Optional citations
- Optional review status
- Optional warning/disclaimer fields

Candidate manifest fields:

```
{
  "id": "wilderness_basics",
  "title": "Wilderness Basics",
  "version": "0.1.0",
  "category": "wilderness",
  "description": "Basic offline wilderness survival guidance.",
  "license": "TBD",
  "source_name": "TBD",
  "source_url": "TBD",
  "last_reviewed": "YYYY-MM-DD",
  "review_status": "unreviewed | community-reviewed | expert-reviewed",
  "files": []
}
```

10.9 User-Created Packs

Question asked:

> Should users be able to add their own packs?

Answer/direction:

- Start with curated packs first.
- Do not initially make uncontrolled user-added packs central.
- User-added packs can be powerful but may create safety/trust issues.
- Add user pack import later with clear warnings, provenance labels, and maybe review status.

Recommended labels:

- Official POSA pack
- Community pack
- User-imported pack
- Unreviewed pack
- Expert-reviewed pack
- Deprecated pack

Important:

User-created packs should never appear equivalent to reviewed official packs.

---

11. Feature Roadmap

11.1 Proposed Phase-Based Build Strategy

User wants:

- 7–8 phases.
- Codex builds phase by phase.
- User tests/iterates after each phase.
- Public testing as soon as feasible.
- Goal: get from nothing to beyond-MVP/polished app in about one month if scope is tight.

Assistant assessment:

- One month is ambitious but possible for a narrowed, polished public alpha/beta.
- Full validation will take longer.
- Public testing is acceptable if expectations are clear.

11.2 Recommended 8-Phase Roadmap

Phase 0 — Repo and Product Skeleton

Goal:

- Create Android repo and project structure.

Deliverables:

- Kotlin Android project
- Open-source license
- README
- CONTRIBUTING
- CODE_OF_CONDUCT
- Issue templates
- Basic app navigation
- Four tabs:
- Map
- Tools
- Guide
- Packs
- Basic POSA branding

Do not add AI yet.

Phase 1 — Local Storage and Core Data Models

Goal:

- Establish local-first architecture.

Deliverables:

- SQLite/Room schema
- Models for:
- notes
- checklists
- checklist items
- gear/tools
- waypoints
- packs
- guide cards
- sources/provenance
- Local CRUD functionality
- Offline-only operation

Phase 2 — Guide and Pack System

Goal:

- Add installable/readable survival content.

Deliverables:

- Pack manifest format
- Starter content pack
- Guide card reader
- Searchable guide content
- Source metadata display
- Pack install/list/delete
- Import local pack file if feasible

MVP content should be basic, safe, and clearly sourced.

Phase 3 — Checklists and Gear Context

Goal:

- Make the app practical immediately.

Deliverables:

- Preloaded survival checklists

- User-created checklists
- Gear/tool inventory
- “I have this” / “I don’t have this” context
- Checklist templates
- Ability to link checklist to guide cards or notes

Important:

Gear inventory should later feed guided chat context.

Phase 4 — Offline Maps and Waypoints

Goal:

- Add real field utility.

Deliverables:

- Offline map viewer
- GPS location display
- Saved waypoints
- Basic waypoint list/details
- Optional breadcrumbs/backtrack if feasible
- Optional GPX import/export if feasible

Unresolved:

- Map library must be selected before implementation.

Phase 5 — Tools Tab

Goal:

- Add field tools.

Deliverables:

- Coordinates display
- Compass if device supports it
- Flashlight control
- Timer
- Unit conversion if simple
- Notes shortcut
- Emergency quick actions
- Battery-conscious UI

Avoid overcomplicating tools.

Phase 6 — Retrieval / Guided Q&A

Goal:

- Add “chat with manuals” without making it an unsafe general chatbot.

Deliverables:

- Search over installed packs
- Guided question interface
- Retrieved source cards
- Answer card template
- Citations/source snippets
- Confidence/provenance UI
- “Based on your gear” contextual answers if possible
- Clear warnings when sources are weak or absent

Initial version may use retrieval-only answers or templated answer composition before a local LLM is added.

Phase 7 — Optional Local AI

Goal:

- Add optional local model support.

Deliverables:

- Model optional download/install
- Small model runtime evaluation
- Battery/thermal guardrails
- Short answer generation
- Summarization
- Source-grounded generation
- No answer without sources unless clearly marked

This phase may slip beyond one month.

Phase 8 — Public Beta / Community Layer

Goal:

- Make it shareable and credible.

Deliverables:

- GitHub release
- F-Droid readiness if feasible
- Documentation site or README

- Screenshots
- Demo video
- Public roadmap
- Community pack guidelines
- Feedback template
- Field testing disclaimer
- Known limitations
- Safety/provenance policy

Delayed from this phase unless simple:

- Bluetooth mesh
- Wi-Fi Direct sharing
- Signed field notes
- Pack registry
- Meshtastic bridge

---

12. Things Considered But Rejected or Delayed

12.1 Multi-OS First

Rejected for now.

Reason:

- Too much complexity.
- Slower to polish.
- Android is more aligned with target users and field devices.
- Open-source Android app is enough initially.

12.2 Python-First App Bridged to Android

Rejected for main app.

Reason:

- Performance/UI/Android integration concerns.
- Possible CPU/thermal/battery problems.
- Native Android likely better for maps, sensors, offline storage, and permissions.

12.3 Bluetooth Mesh in MVP

Delayed.

Reason:

- Not necessary for first value.
- Complex.
- Could distract from core features.

12.4 Full Meshtastic Integration

Delayed.

Reason:

- Hardware dependency.
- Complicates consumer onboarding.
- Better as optional later feature.

12.5 User-Created Packs From Day One

Delayed / limited.

Reason:

- Safety and provenance issues.
- Curated packs should come first.
- Later support with clear warnings and labels.

12.6 Heavy Local LLM as MVP Requirement

Delayed.

Reason:

- Can slow development.
- Hardware-sensitive.
- App should be useful without AI.
- Retrieval and provenance should come first.

12.7 Government-Specific Features in Public POSA

Do not include initially.

Reason:

- Avoid giving away moat.
- Avoid confusing public survival app with government platform.
- Keep ATAK/SOP/government workflows commercial/private.

13. Constraints and Assumptions

13.1 Offline-First Constraint

The app should be useful with:

- No account
- No cloud
- No internet
- No cellular signal
- No backend requirement

Optional future cloud/community registry is allowed, but core use must stay offline.

13.2 Free/Open Source Constraint

The public POSA app should be:

- Free
- Open source
- Community-friendly
- Easy to inspect
- Easy to install if possible

Potential distribution:

- GitHub releases
- F-Droid eventually
- Google Play maybe later

13.3 Trust/Safety Constraint

Because this is a survival app:

- Do not present generated content as guaranteed correct.
- Show sources.
- Separate source text from synthesis.
- Mark unreviewed content.
- Avoid dangerous overconfidence.
- Avoid unsupported medical claims.
- Add disclaimers where appropriate.
- Keep first-aid guidance especially careful.

13.4 Content Constraint

The app should avoid becoming a random PDF dump.

Content should be:

- Curated
- Licensed appropriately
- Versioned
- Cited
- Chunked/indexed cleanly
- Reviewed when possible
- Provenance-tagged

13.5 Performance Constraint

Target devices may include older Android phones.

Need to watch:

- App size
- Map data size
- Content pack size
- Battery drain
- Thermal throttling
- Memory footprint
- Offline indexing time
- Search latency
- Model load time if optional AI is added

13.6 Scope Constraint

Do not try to build:

- Trail Sense
- Organic Maps
- Kiwix
- SurvivalRAG
- Meshtastic
- Briar

all at once.

POSA should integrate selected features into one practical field app.

---

14. Open Questions / Unresolved Decisions

14.1 Map Stack

Need to decide:

- MapLibre?
- Mapsforge?
- MBTiles?
- PMTiles?
- Organic Maps-derived approach?
- OsmAnd components?
- Another Android offline map library?

Need evaluate:

- License compatibility
- Offline vector map support
- Topographic/contour support
- GPX support
- APK size
- Runtime performance
- Ease of Codex implementation
- Long-term maintainability

14.2 Android Tech Stack

Need final decision:

- Kotlin native Android
- Jetpack Compose
- Room/SQLite
- Map library TBD

Likely best default:

- Kotlin
- Jetpack Compose
- Room
- SQLite FTS
- MapLibre or similar after research

14.3 License

Need choose license.

Options discussed from ecosystem:

- Apache-2.0
- MIT
- GPL-style reciprocal license

Recommendation trend:

- App core: Apache-2.0 if wanting adoption/partner reuse.
- Content packs: separate licenses per pack.
- Pack schema/protocol: open spec.

Need final decision from user.

14.4 Content Sources

Need define initial starter packs.

Possible packs:

- Wilderness basics
- Water
- Fire
- Shelter
- Navigation
- Signaling
- First-aid quick cards
- Urban emergency prep
- Vehicle emergency
- Bug-out / home prep

Need verify source licensing.

Do not assume manuals are legally reusable without checking.

14.5 Local AI Runtime

Need decide later:

- Whether to support GGUF on Android
- Which runtime
- Whether model runs in-app or via optional local server
- Whether to delay generation entirely

MVP can avoid this.

14.6 User Pack Import

Need decide:

- No user packs in MVP
- Import local packs in MVP but mark as unreviewed
- Only official packs in MVP

Recommended:

- Curated official starter packs first.
- User import later or behind “advanced” mode.

14.7 Analytics / Usage Tracking

User wants to track usage for traction/fundraising.

But app is offline/open-source.

Need consider privacy-preserving options:

- No analytics by default.
- Optional telemetry opt-in.
- GitHub stars/downloads as public metrics.
- F-Droid/Play install counts if distributed.
- In-app “send feedback” export.
- Public issue templates.
- Anonymous optional crash reports only if users consent.

Important:

Do not betray offline/privacy positioning by silently tracking users.

14.8 Commercial Boundary

Need decide what lives in:

- POSA public repo
- Unplugged AI private repos
- Future commercial/government platform

Codex should avoid accidentally placing proprietary/government logic into the public POSA repo.

---

15. Recommended Initial Codex Build Prompt

Use this as the first Codex task after repo setup:

```
Build an Android-first Kotlin app called POSA — Portable Offline Survival
Assistant.

Goal:
Create a free/open-source offline-first survival companion built by Unplugged
AI.

Initial scope:
- Android only.
- Kotlin native app.
- Four tabs: Map, Tools, Guide, Packs.
- Fully offline-first.
- No account, no cloud dependency.
- No AI model in MVP.
- Use local SQLite/Room storage.

Core data:
- Notes
- Checklists
- Gear/tool inventory
- Waypoints
- Content packs
- Guide cards
- Source/provenance metadata

Features:
1. Guide tab:
   - Show installed guide cards.
   - Search guide content locally.
   - Each card displays title, category, source, pack, license, version, and
last reviewed date.

2. Packs tab:
   - Show installed packs.
   - Include one bundled starter pack.
   - Define a pack manifest schema.
   - Allow future local pack import, but keep MVP simple.

3. Checklists:
   - Include starter survival checklists.
   - Allow users to create/edit/check off items.
   - Store locally.
```

```
4. Gear/tools inventory:
   - Let users add gear/tools they have.
   - Store locally.
   - This will later be passed into guided Q&A context.

5. Tools tab:
   - Coordinates display placeholder or implementation if permission available.
   - Timer.
   - Notes shortcut.
   - Flashlight placeholder or implementation if simple.
   - Compass placeholder or implementation if simple.

6. Map tab:
   - Current location display.
   - Saved waypoints.
   - Offline map integration can start as placeholder if library selection is
not finalized.
   - Structure code so map provider can be swapped later.

7. Provenance:
   - Make provenance visible across guide content.
   - Separate content/source metadata from user notes and future AI-generated
answers.

Repo:
- README.md
- LICENSE placeholder
- CONTRIBUTING.md
- docs/pack-schema.md
- docs/roadmap.md
- .gitignore
- GitHub issue templates if feasible

Non-goals:
- Do not add Bluetooth mesh.
- Do not add Meshtastic.
- Do not add cloud backend.
- Do not add user accounts.
- Do not add government/ATAK features.
- Do not add heavy local LLM yet.
- Do not build iOS or desktop.
```

---

16. Immediate Next Steps

16.1 Decide Repo Basics

Need user decision:

- GitHub org/repo:
- likely `unpluggedai/posa`
- License:
- likely Apache-2.0 unless stronger copyleft desired
- Public or private during initial build:
- likely private until first usable alpha, then public

16.2 Decide Android Stack

Recommended default:

- Kotlin
- Jetpack Compose
- Room/SQLite
- SQLite FTS
- Offline map library TBD

Need Codex research task for map library.

16.3 Create Phase 0 Issue List

Suggested GitHub issues:

1. Initialize Android project
2. Add four-tab navigation
3. Add local database schema
4. Add starter pack manifest
5. Add bundled guide cards
6. Add checklist module
7. Add gear inventory module
8. Add notes module
9. Add waypoint model/UI
10. Research offline map library
11. Add provenance card component
12. Add README and contribution docs

16.4 Build Starter Content Pack

Need create:

```
packs/
  wilderness-basics/
    manifest.json
    cards/
      water.md
      fire.md
      shelter.md
      navigation.md
      signaling.md
      emergency-priorities.md
```

Manifest should include:

- source metadata
- license metadata
- review status
- version

Content should be conservative and clearly marked if not expert-reviewed.

16.5 Prepare Public Messaging

Possible README intro:

```
# POSA — Portable Offline Survival Assistant

POSA is a free, open-source offline survival companion built by Unplugged AI.

It is designed for hikers, campers, preppers, responders, and off-grid users who
need maps, field tools, checklists, notes, and source-grounded survival
knowledge without relying on cloud services or cell signal.
```

16.6 Define Safety Policy

Need document:

- Not medical/legal/emergency professional advice.
- Use official emergency services when available.
- First-aid content must be sourced and reviewed carefully.
- AI-generated answers must cite sources.
- Unreviewed packs clearly marked.
- User-imported packs are not endorsed.

---

17. Key Warnings for Codex

Codex should avoid these mistakes:

17.1 Do Not Build a Generic Chatbot

The product is not “ChatGPT for survival.”

It is an offline field companion with guided source-grounded assistance.

17.2 Do Not Make AI Required

The app must be useful without a model installed.

Search, guides, maps, checklists, notes, and tools should work independently.

17.3 Do Not Hide Sources

Every guide card and future generated answer should expose provenance.

17.4 Do Not Add Cloud Dependencies

No account, no required backend, no forced analytics.

17.5 Do Not Overbuild Mesh/Comms

Bluetooth mesh, Meshtastic, Wi-Fi Direct, and LAN sync are later features.

17.6 Do Not Build Multi-Platform First

Android first.

Do not start with desktop, iOS, or cross-platform unless user explicitly changes direction.

17.7 Do Not Leak Moat

Do not include:

- government SOPs
- ATAK integrations
- proprietary model optimization
- evaluation harness
- customer-specific workflows

- Unplugged AI commercial platform logic

in the public POSA repo.

17.8 Do Not Treat User Packs as Trusted

User-imported/community packs need labels and warnings.

Official curated packs should be distinct.

17.9 Do Not Assume Public Validation Means Production Safety

The user wants public testing, but POSA should clearly mark itself as evolving/alpha/beta until mature.

---

18. Fundraising / Business Narrative

POSA should support future fundraising by showing:

- Bootstrapped execution
- Public repo activity
- GitHub stars
- Downloads
- User feedback
- Field testing
- Community adoption
- Practical offline-first edge capability
- Clear path from civilian app to government/commercial edge intelligence

Narrative:

> We built POSA as the free/open-source public proof point for our offline edge-intelligence architecture. The same architecture powers Unplugged AI's hardened decision-grade systems for government, field operations, emergency response, and industrial safety.

---

19. Government Pilot Narrative

For government buyers, POSA is not the deliverable.

POSA proves:

- Offline-first UX
- Local content packs

- Source-grounded answers
- Field usability
- Android/rugged device path
- Edge deployment discipline

Government/commercial version adds:

- Approved SOP packs
- Mission-specific workflows
- ATAK integration
- Secure pack distribution
- Evaluation/testing
- Device deployment
- Admin controls
- Support
- Custom local model optimization

Potential pilot angles:

1. ATAK SOP Assistant
2. Offline training companion
3. Edge knowledge pack system
4. Contested-comms decision support
5. Disaster response / CERT preparedness assistant
6. Border Patrol field procedure assistant
7. SAR/responder offline guide system

---

20. One-Month Build Reality

User believes a strong beyond-MVP/polished public version can be built in about one month using Codex phase-by-phase.

Assistant guidance:

- One month is possible for a polished public alpha/beta if scope stays tight.
- Full validation takes longer and should happen publicly over time.
- Public testing is acceptable if transparent:
- “This is new.”
- “We are testing in the field with the community.”
- “Do not rely on it as sole source of emergency guidance.”
- “Report issues and contribute.”

Do not promise a complete best-in-market app in one month.

Better target:

- Month 1: polished Android public alpha with maps/checklists/guides/tools/packs/provenance skeleton.
- Month 2–3: retrieval, better maps, user feedback, field testing, optional AI.
- Month 6+: serious category contender.

---

21. Current Final Plan

The current plan being converged toward:

```
Build POSA as an Android-first, free/open-source survival app.

Start with:
- Offline maps
- Checklists
- Tools
- Gear/tool inventory
- Field notes
- Curated survival guide packs
- Provenance/source display
- Guided manual Q&A / retrieval

Delay:
- Bluetooth mesh
- Meshtastic
- LAN sync
- P2P pack sharing
- iOS
- Desktop
- Heavy local LLM
- Government-specific features

Use POSA to:
- Build community
- Get GitHub stars
- Get field feedback
- Prove Unplugged AI execution
- Strengthen government/fundraising narrative

Keep Unplugged AI moat in:
- faster/smaller proprietary edge systems
- custom government/SOP packs
- ATAK/government workflows
```

```
- evaluation harness
- optimized local inference
- deployment/support
```

---

22. Short Codex Summary

If Codex needs the shortest possible summary:

```
We are building POSA — Portable Offline Survival Assistant — a free/open-source
Android-first offline survival app built by Unplugged AI.

It should start as a polished Android app with four tabs: Map, Tools, Guide,
Packs. MVP features are offline maps, saved waypoints, checklists, gear/tool
inventory, field notes, curated survival content packs, local search/guided Q&A
with manuals, and visible provenance/citations.

Do not build a generic chatbot. Do not require cloud. Do not require AI. Do not
add Bluetooth mesh, Meshtastic, iOS, desktop, ATAK, government SOPs, or heavy
local LLM in the MVP.

Use Kotlin/native Android if possible, SQLite/Room for local storage, SQLite FTS
for initial search, and design the pack/retrieval system so optional local RAG
and local models can be added later.

Open-source the public survival app and basic pack system. Keep Unplugged AI's
moat private: advanced model optimization, evaluation harness, government/ATAK
integrations, custom SOP packs, secure deployment, and rugged edge intelligence
platform.

Goal: build a useful public field app that outdoor/prepper users adopt and
share, while positioning Unplugged AI as the company behind decision-grade
intelligence at the edge.
```